SQL_TEST_01 · COMMAND REFERENCE · LIVING DOCUMENT

Everything you've typed so far

Every command, meta-command, and function actually used hands-on across this project — grouped by tool, in the order you met them, not alphabetized — plus a project-wide file map, and a running list of Claude's own working commands. Appended at the end of each build, not written once and frozen.

COVERS

Build 1 (agent), Build 2 (pipeline), Build 2.5 (scratch workspace) & Build 3 (semantic search) — all complete

ENVIRONMENT

Windows 11 · PowerShell · Docker Desktop

DATABASES

pg_local (appdb)

Project Files

WHAT'S ACTUALLY IN THIS FOLDER

A map of the project root as of Build 3.5's start — what each file is for, not what commands it contains (that's every section below). Excludes auto-generated folders (`.venv/`, `__pycache__/`, `.git/`).

SQL migrations — run in order, numbered for reproducibility

`001_schema.sql`

Original appdb schema — `customers`/`orders`, the seed data every later build extends.

`002_seed.sql`

Idempotent seed data for `customers`/`orders`.

`003_readonly_role.sql`

`appdb_reader` — the read-only role Build 1's agent runs as.

`004_raw_schema.sql`

Build 2 Phase 1 — `raw.allocations`, all-`TEXT` ingestion target.

`005_clean_schema.sql`

Build 2 Phase 2 — `clean.allocations`, real types plus the computed `duration_seconds` column.

`006_agent_scratch_role.sql`

Build 2.5 Phase 1 — `appdb_agent_writer`, the agent's write-scoped role.

`007_agent_scratch_schema.sql`

Build 2.5 Phase 1 — the `agent_scratch` schema and its grants.

`008_add_summary_embedding.sql`

Build 3 Phase 1 — adds `clean.allocations.summary_embedding vector(384)`.

`009_add_summary_embedding_hnsw_index.sql`

Build 3 follow-up — HNSW index on `summary_embedding` for indexed similarity search.

`schema_mssql.sql`

The SQL Server counterpart to `001_schema.sql` — same appdb shape, T-SQL dialect.

`seed_mssql.sql`

SQL Server counterpart to `002_seed.sql`.

Python — the agent itself

`agent.py`

The hand-built tool-use loop — calls the Anthropic API, dispatches tool calls, enforces `MAX_TOOL_TURNS`.

`tools.py`

Every tool the agent can call: `run_sql_query`, `list_tables`, `describe_table`, `save_dataframe`, `search_summaries`.

Python — data pipeline (Build 2)

`ingest.py`

Loads the source CSV into `raw.allocations`, one `executemany` call.

`profile_raw_data.py`

Pandas exploration of `raw.allocations` that informed `005_clean_schema.sql`'s design. Renamed from `profile.py` — it silently shadowed Python's stdlib `profile` module.

`transform.py`

Raw → clean transform: type coercion, blank→NULL handling, populates `clean.allocations`.

Python — semantic search (Build 3)

`embed_summaries.py`

Idempotent backfill — embeds every `summary` row still missing a `summary_embedding`.

`search_summaries.py`

Standalone script proving raw cosine-distance search works, before any agent wiring.

`calibrate_threshold.py`

Statistical calibration of the 0.63 relevance-distance cutoff, from 40 curated in-/out-of-domain queries.

Python — early sanity checks (pre-Build-1)

`db_test_py.py`

The very first Postgres connectivity check — plain `psycopg`, no agent involved.

`api_test.py`

The very first Anthropic API call — confirms auth and a real response, before any tool-use loop existed.

Tests

`test_data_quality.py`

Build 2 — `pandera` schema checks against `clean.allocations`.

`test_agent_scratch_boundary.py`

Build 2.5 — role-boundary tests plus the `MAX_SCRATCH_TABLES` cap test.

`test_semantic_search.py`

Build 3 — embedding coverage, relevant/irrelevant query behavior, HNSW index existence.

Configuration & infrastructure

`docker-compose.yml`

Defines `pg_local`, `mssql_local`, and the `agent` service; Postgres image is `pgvector/pgvector:pg17`.

`Dockerfile`

Builds the `agent` service's image.

`requirements.txt`

Pinned Python dependencies — `anthropic`, `psycopg`, `pgvector`, `sentence-transformers`, `pandas`, `pandera`, `pytest`, `python-dotenv`.

`.env`

Real local secrets (DB passwords) — gitignored, never committed.

`.env.example`

Template shape of `.env` with placeholder values, safe to commit.

`.gitignore`

Excludes secrets (`.env`, `CREDENTIALS.local.md`), OS/editor cruft, `__pycache__/`, logs/dumps.

`.gitattributes`

Forces consistent line endings for `.sql/.yml/.md` across Windows/Unix.

Docs, secrets & generated artifacts

`CREDENTIALS.local.md`

Unredacted local companion to `.env` — gitignored, never shared or committed.

`sql-test-01-cheatsheet.html / .pdf`

This living document — every command actually typed, plus this file map.

`PROJECT_DIAGRAMS/`

One subfolder per build (`BUILD_1_FLOW` … `BUILD_3_5_FLOW`) — flow diagrams (`.svg/.html/.png`) and handoff briefs (`.html/.pdf`).

`Local Postgres.session.sql`

Empty scratch file created by a VS Code SQL client extension — not authored content.

`.vscode/, .pytest_cache/`

Editor settings and pytest's own cache — auto-generated, not source.

Docker

CONTAINER ORCHESTRATION

Starting the stack, and getting SQL in and out of a running container.

```
docker compose up -d --build
```

Build images if anything changed, then start every service in the background.

```
docker ps
```

List running containers — the first check when a connection unexpectedly fails.

```
docker exec -it pg_local psql -U devuser -d
appdb
```

Open a **live, interactive** psql session inside the Postgres container.

```
Get-Content file.sql -Raw | docker exec -i
pg_local psql -U devuser -d appdb
```

Run a whole .sql file inside the container **non-interactively**, PowerShell-style — < redirection doesn't exist in PowerShell, so the file gets piped in instead. (Bash/cmd would use `docker exec -i ... < file.sql`.)

```
docker compose down -v
```

Stop everything and delete volumes too — next start re-initializes DB passwords from .env. Used deliberately, not casually.

psql

META-COMMANDS & DDL PATTERNS

Once you're inside a psql session — both the backslash meta-commands and the recurring SQL shapes.

```
\d table_name
```

Describe a table — every column, its type, and any constraints. The one check that settles "did my CREATE TABLE actually work."

```
\du role_name
```

Describe a role — confirms it has no Superuser/Createrole/Createdb it shouldn't.

```
\pset null 'marker'
```

Display NULLs as a visible marker instead of blank — otherwise a NULL and an empty string look identical.

```
\! cls
```

Shell out and run an OS command without leaving psql — `cls` clears the terminal screen.

```
\q
```

Quit psql, back to the shell prompt.

```
CREATE SCHEMA IF NOT EXISTS name;
```

Create a schema, skip quietly if it's already there — safe to rerun.

```
DROP TABLE IF EXISTS name;
```

Drop a table, skip quietly if it doesn't exist — what makes a rebuild script rerunnable from a blank database.

```
GRANT SELECT ON ALL TABLES IN SCHEMA s TO
role;
```

Give a role read access to every table **currently** in a schema.

```
ALTER DEFAULT PRIVILEGES IN SCHEMA s GRANT
SELECT ON TABLES TO role;
```

Extend that same grant automatically to any table created **later** — without this, new tables start private.

```
col TYPE GENERATED ALWAYS AS (expr) STORED
```

A column Postgres computes and locks from other columns in the same row — structurally impossible for it to drift out of sync.

```
GRANT USAGE, CREATE ON SCHEMA s TO role;
```

Let a role create objects inside a schema, not just read from it — how `appdb_agent_writer` got write access scoped to `agent_scratch` only, never anywhere wider.

```
ALTER DEFAULT PRIVILEGES FOR ROLE owner IN
SCHEMA s GRANT SELECT ON TABLES TO role;
```

Extend a **different** role's read access to tables a specific owner role creates later in that schema — how `appdb_reader` can always see whatever `appdb_agent_writer` saves, without a re-grant each time.

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Enable pgvector for the database — requires the `pgvector/pgvector` Postgres image, not the plain `postgres` one.

```
ALTER DATABASE db REFRESH COLLATION VERSION;
```

Clear the collation-version mismatch warning that appears after swapping to an image built against a different glibc/ICU — safe here since the data is plain ASCII text.

```
col vector(384)
```

pgvector's column type for a fixed-dimension embedding — 384 matches the sentence-transformers model in use, not an arbitrary choice.

```
a ⟺ b
```

Cosine distance between two vectors — smaller means more similar. What `search_summaries` orders by.

```
a ⟷ b
```

Euclidean (L2) distance between two vectors — used in this project only for the self-distance sanity check (`v <-> v` should be exactly 0).

```
vector_dims(col)
```

The dimensionality of a stored vector — a quick sanity check that every row actually has a real 384-dim embedding, not a malformed one.

```
CREATE INDEX ... USING hnsw (col
vector_cosine_ops)
```

Build an approximate-nearest-neighbor graph index matching the `<=>` operator, so similarity search can use an index scan instead of a full table scan as the corpus grows.

```
EXPLAIN ANALYZE SELECT ...
```

Confirms whether the planner actually used a new index (`Index Scan`) or fell back to `Seq Scan` — the only real way to verify an index is doing anything.

PowerShell

HOST-SIDE SHELL

Running Python, checking your own environment, and vetting/installing system-level tools from outside any container.

```
.\.venv\Scripts\python.exe script.py
```

Run a script with the project's own virtual-env interpreter directly — sidesteps PATH/activation issues entirely.

```
.\.venv\Scripts\python.exe -m pip install pkg
```

Install a package into that specific virtual env, not wherever PATH happens to point.

```
.\.venv\Scripts\python.exe -m pip show pkg
```

Check whether a package is installed, and exactly which version.

```
Get-ChildItem Env: | Where-Object { $_.Name -
like "*X*" }
```

List environment variables whose name matches a pattern — e.g. catching a stray API key leaked into the shell.

```
Get-Content file
```

Print a file's contents — the go-to way to verify what actually got saved to disk.

```
claude auth status
```

Confirm this Claude Code session is on subscription login, not silently metered API billing.

```
winget show --id pkg -e
```

Pull a package's full manifest — publisher, source URL, license, install hash — before trusting or installing it.

```
winget install --id pkg -e
```

Install a package system-wide once it's been vetted, not before.

Git

CHECKPOINTING PROGRESS

Committing at the end of every phase, per the project's own working style.

`git status`

What's changed, staged, or untracked, right now.

`git add file`

Stage a specific file by name — never a blind `-A`.

`git commit -m "..."`

Record the staged snapshot with a message explaining **why**, not just what.

`git mv old new`

Rename a tracked file while git keeps its history attached to the new name.

`git log --oneline -n`

Recent commit history, one line each — the fastest way to check "where are we, really."

`git diff`

Exact unstaged changes, line by line, before they get staged.

Python — psycopg

TALKING TO POSTGRES

The driver behind every script in this project — agent.py, tools.py, ingest.py, profile.py.

```
psycopg.connect(host=, dbname=, user=,
password=)
```

Open a connection to a Postgres database.

```
with conn.cursor() as cur:
```

Open a cursor scoped to a `with` block — closes itself automatically, even on error.

```
cur.execute(sql, params)
```

Run one SQL statement, with parameters passed safely (never string-formatted into the query).

```
cur.executemany(sql, rows)
```

Run the same statement once per row — how ingest.py loads 1,535 rows in one call.

```
cur.fetchall()
```

Pull every result row back as a Python list.

```
cur.description
```

Metadata about the result's columns — used to grab column names generically.

```
conn.commit()
```

Make written changes permanent.

```
conn.close()
```

Release the connection.

Python — psycopg.sql

SAFE DYNAMIC IDENTIFIERS

save_dataframe's guardrail for building a CREATE TABLE/INSERT statement where the table and column names themselves come from the model, not just the values — %s parameters alone can't protect an identifier.

`from psycopg import sql`

The submodule that builds SQL safely out of parts psycopg can't treat as plain data — like table and column names.

`sql.Identifier(name)`

Safely quotes a dynamic identifier (table/column name) — the equivalent of a parameter, but for names instead of values.

`sql.SQL(" … {} … ").format( … )`

Builds a SQL string from literal fragments plus safely-escaped pieces like `Identifier` — never plain `f"..."` string interpolation.

`sql.SQL(", ").join(parts)`

Joins a list of `Identifier`/`SQL` fragments with a separator — how a variable-length column list gets assembled.

`sql.Placeholder()`

A %s value placeholder built the same composable way, for when the number of columns isn't known until runtime.

Python — pandas

PROFILING RAW DATA

Everything profile_raw_data.py used to understand raw.allocations before designing the clean schema. (Renamed from profile.py in Build 3 — it was silently shadowing Python's stdlib profile module.)

`pd.DataFrame(rows, columns=cols)`

Build a DataFrame straight from plain rows + column names — no SQLAlchemy required.

`df["col"].unique()`

Every distinct value in a column, once each — printed with `repr()` to expose hidden whitespace.

`df["col"].value_counts(dropna=False)`

Count of each distinct value, including blanks/NaN.

`pd.to_datetime(s, format=, errors="coerce")`

Parse a column of date strings against an exact format; anything that doesn't match becomes `NaT` instead of crashing the run.

`series.dt.total_seconds()`

Convert a time difference into a plain number of seconds.

`series.str.fullmatch(pattern)`

Test every value in a column against a regex at once — used to confirm two columns were digit-only.

Python — pandera

DATA-QUALITY SCHEMAS

test_data_quality.py uses this to declare what "correct" looks like for clean.allocations, as data instead of ad hoc checks.

`import pandera.pandas as pa`

The explicit pandas-backend import for modern pandera versions.

`pa.DataFrameSchema({ ... })`

Declare an entire DataFrame's expected shape — one entry per column — as data, not scattered asserts.

`pa.Column(type, checks, nullable=)`

One column's expected type, whether NULLs are allowed, and any constraints.

`pa.Check.ge(0)`

A reusable constraint — here, "greater than or equal to 0" — attached to a Column.

`schema.validate(df)`

Check a real DataFrame against the schema; raises a real error the moment something doesn't match.

pytest

RUNNING THE TEST SUITE

First automated test suite in the project — codifies checks that were previously done by hand.

`.\.venv\Scripts\python.exe -m pytest -v`

Discover and run every `test_*` function in the project, printing each one's name and pass/fail individually.

`def test_something():`

Any function named `test_...` in a `test_*.py` file is auto-discovered — no registration needed.

`assert condition`

Plain Python `assert` — pytest reports exactly which assertion failed and the actual vs. expected values.

`with pytest.raises(ExceptionType):`

Asserts a block **must** raise a specific exception — used to prove a role boundary actually blocks a write, not just that it "seems fine."

```
def test_x(monkeypatch):
monkeypatch.setattr(mod, "NAME", val)
```

Temporarily overrides a module-level constant for one test only, auto-restored after — how a cap like `MAX_SCRATCH_TABLES` gets tested at its boundary without creating 50 real tables.

pgvector & embeddings

SENTENCE-TRANSFORMERS · PGVECTOR.PSYCOPG

Turning free text into vectors and getting Postgres to store/compare them — `embed_summaries.py`, `search_summaries.py`, `tools.py`'s `search_summaries`.

```
from sentence_transformers import
SentenceTransformer
```

A local, free embedding model library — no API key or per-call cost, unlike Voyage AI/OpenAI embeddings.

```
SentenceTransformer("all-MiniLM-L6-v2")
```

Loads the specific 384-dim model used throughout this project — first call downloads and caches it locally.

```
model.encode(text_or_texts,
show_progress_bar=True)
```

Turns one string or a list of strings into embedding vector(s) — a progress bar matters when embedding 1,500+ rows at once.

```
from pgvector.psycopg import register_vector
```

Teaches a psycopg connection how to send/receive pgvector's `vector` type directly as numpy-like arrays, instead of raw strings.

```
register_vector(conn)
```

Applies that registration to a specific connection — needed once per connection, before any vector query runs.

```
cur.execute("... WHERE col ⟺ %s < %s ...",
(embedding, threshold))
```

Passes a computed embedding straight in as a query parameter — the distance operator and the threshold cutoff both happen inside the SQL, not in Python after fetching everything.

Python — standard library

CSV · OS · DOTENV

No extra install required — these ship with Python itself, or python-dotenv.

`csv.DictReader(f)`

Read a CSV where each row comes back as a `{column: value}` dict — chosen over pandas for ingestion specifically so nothing gets silently type-coerced.

`open(path, newline="", encoding="utf-8-sig")`

Open a file safely for CSV reading, transparently stripping a leading BOM if present.

`os.environ["KEY"]`

Read a required env var — raises immediately if it's missing, rather than continuing with a silent gap.

`os.environ.get("KEY", default)`

Read an optional env var, falling back to a default (e.g. `POSTGRES_HOST` defaulting to `127.0.0.1` outside Docker).

`load_dotenv(encoding="utf-8-sig")`

Load .env into the process environment — BOM-safe, matching how Windows tools often save that file.

Claude's Commands

DIAGRAMS · VERIFICATION · PUBLISHING

A separate running list from everything above — not what *you've* typed, but the commands Claude actually runs behind the scenes while building diagrams, verifying renders, and publishing docs for this project. Appended to as new patterns come up, same as every other section.

```
msedge --headless --disable-gpu --window-
size=W,H --screenshot=out.png
file:///path.html
```

Renders an HTML file exactly as a browser would and saves it as a PNG — the first of two required checks before any diagram or doc is considered verified.

```
msedge --headless --disable-gpu --print-to-
pdf=out.pdf --print-to-pdf-no-header --no-pdf-
header-footer file:///path.html
```

Renders the same page through Chromium's separate print pipeline — a genuinely different code path that has caught real bugs (`<img>` references silently failing to render) the screenshot path alone missed.

```
pdftoppm -png -r 100 file.pdf prefix
```

Rasterizes every page of a generated PDF into numbered PNGs, so each page can actually be looked at instead of trusting file size alone.

```
pdfinfo file.pdf | grep -i pages
```

Quick page-count check before rasterizing — confirms whether a doc paginated the way it was expected to.

```
git commit -m "$(cat <<'EOF' ... EOF)"
```

Heredoc commit-message convention — explains **why** a change happened in the body, and avoids hand-escaping quotes/newlines. Every commit message ends with a `Co-Authored-By` line.

```
Image.open(path).convert("RGB").getpixel((x,
y))
```

Samples an exact on-screen pixel's RGB value with Pillow — confirms a color rendered exactly as CSS specified, rather than trusting a visual impression at small/compressed scale. Caught a false alarm (anti-aliasing fringe mistaken for a real color bug) during this cheat sheet's own verification.

Living document — appended at the end of each build, not written once and frozen. Reflects hands-on usage through Build 3, including its follow-up hardening pass (idempotent re-embedding, HNSW index) — all complete. Not exhaustive documentation for any of these tools, just what's actually been typed in this project so far.